

Aurviz

A personal AI agent for Android

Project Report

Requirements · Architecture · Testing · Performance · Guides

13 November 2026

Contents

1. Software Requirements Specification
2. Architecture
3. Test Report
4. Performance
5. Evaluator Guide
6. User Guide
7. Backend API
8. Build and Deployment

Software Requirements Specification

Product: Aurviz, a personal AI agent for Android **Version:** 1.1 (November 2026) **Status:** Implemented and verified (see Test Report)

---

1. Introduction

1.1 Purpose

This document specifies what Aurviz must do (functional requirements) and how well it must do it (non-functional requirements). Every requirement has an ID, an acceptance criterion and a test, so that a reviewer can verify it directly.

1.2 Scope

Aurviz is an Android application, backed by a small Node.js service, that understands spoken and typed instructions, remembers the user's preferences, plans multi-step work, calls tools (Gmail, memory, scheduler, web browser) and executes tasks, including scheduled tasks that run when the app is closed. It is more than a chat bot: the model decides *what* to do, Kotlin code *does* it, and WorkManager decides *when*.

1.3 Definitions

Term	Meaning
Agent	The orchestration loop that asks Gemini for the next step, runs the tool it picks, and feeds the result back
Tool	A Kotlin class implementing `AgentTool` that performs one operation (for example `search_emails`)
Sensitive tool	A tool with an external side effect (sending mail, clicking in a web page). It always needs explicit user approval
Demo mailbox	A built-in sample inbox that lets the app be evaluated without any Google account
HITL	Human in the loop: the approval dialog shown before a sensitive action

1.4 Responsibilities (three separate concerns)

Concern	Owner
Reasoning: decide what to do next	Gemini (function calling)
Execution: perform the operation	Kotlin tools and the backend
Scheduling: run later	Android WorkManager

Gemini cannot wake the app. A scheduled task is persisted in Room and registered with WorkManager, which later invokes the same agent loop in background mode.

---

2. Overall description

2.1 Users

- **End user:** a person who wants an assistant for email, reminders and quick web tasks.
- **Evaluator:** a reviewer who must be able to test every feature without receiving anyone's credentials (see Evaluator Guide).

2.2 Operating environment

Android 8.0 (API 26) or newer, internet access, a Gemini API key (bundled at build time or entered in Settings). Gmail features need a connected Google account **or** the demo mailbox.

2.3 Constraints

- WorkManager periodic work is durable but not exact to the minute (Doze and battery policy apply).
- Google restricts the `gmail.readonly` scope: accounts must be listed as OAuth test users until the app passes Google verification.
- The controlled browser only opens public HTTPS pages.

2.4 Assumptions

The backend is reachable over HTTPS. The Gemini free tier may rate-limit requests; the app handles this (retry, model fallback).

---

3. Functional requirements

Priority: **P0** must have, **P1** should have, **P2** nice to have.

3.1 Core requirements

ID	Requirement	Pri	Acceptance criterion	Test
FR-01	**Voice input.** Accept spoken commands and transcribe them with Android speech recognition	P0	Tapping the microphone, speaking, and the transcribed text appears as a "Voice" message and is processed	Device (earlier build); `VoiceManager`
FR-02	**Text input.** Accept typed commands	P0	A typed prompt receives an answer	`AgentOrchestratorTest`; device
FR-03	**Intent detection.** Recognise what the user wants and which tool is needed	P0	"Check my emails" triggers `search_emails`; "draft, don't send" triggers `create_email_draft` and never `send_email`	`AgentReliabilityTest`; device
FR-04	**Tool calling.** Invoke tools with structured arguments and return results to the model	P0	The timeline shows each tool call with status and result	`AgentOrchestratorTest`, `GeminiAgentClientTest`; device

ID	Requirement	Pri	Acceptance criterion	Test
FR-05	**Persistent memory.** Save facts and preferences that survive restarts	P0	Save a memory, force-stop the app, reopen, the memory is still listed	Device; `MemoryToolTest`
FR-06	**Memory retrieval.** Use saved memories automatically	P0	After "I prefer formal emails", a later draft is formal	`AgentReliabilityTest.savedPreference…` ; device
FR-07	**Memory edit and delete.** View, edit and delete memories	P0	Edit and delete work from the Memory screen (with confirmation) and by voice ("forget my…")	`MemoryToolTest` ; device
FR-08	**Email sending with confirmation.** Send only after the user approves recipient, subject and body	P0	An editable approval dialog appears; cancelling sends nothing; approving sends once	`AgentOrchestratorTest.processUserPrompt_sensitiveTool…` ; device
FR-09	**Web automation.** Open pages, read, scroll, navigate and extract information	P1	"Open news.ycombinator.com and tell me the top 3 stories" opens the site and lists stories	`ControlledWebScraperTest` , `BrowserUrlTest` ; device
FR-10	**Task scheduling.** Create one-time and daily recurring tasks from natural language	P0	"Every morning at 9 AM…" creates a task showing "Daily at 09:00" and the next run time	`SchedulerToolTest` , `TimeUtilsTest` ; device
FR-11	**Background execution.** Run tasks without the app open	P0	A task run through WorkManager records a result and posts a notification	`TaskExecutionPolicyTest` ; device
FR-12	**Multi-step execution.** Complete several operations in sequence	P0	"Check emails, find interview invitations and summarize them" runs search, then read, then summary	`AgentReliabilityTest` ; device

ID	Requirement	Pri	Acceptance criterion	Test
FR-13	**Task history.** Show previous runs	P0	The History tab lists each run with status, time, duration and attempt	Device
FR-14	**Error reporting.** Show useful errors and recovery actions	P0	Failures show a plain message and a "Try again" action for transient errors	`AgentReliabilityTest`, `ExecutionLogFormatterTest`; device

3.2 Extended requirements (beyond the brief)

ID	Requirement	Pri	Acceptance criterion	Test
FR-15	**Email search and read.** Search a mailbox with Gmail operators and read a message	P0	`subject:`, `from:`, `is:unread`, `newer_than:` filters work	`DemoMailServiceTest`, `gmail-inbox.test.js`
FR-16	**Email drafts.** Save a draft without sending	P0	The draft is stored (Gmail Drafts or demo Drafts) and nothing is sent	`GmailInboxToolsTest`; device
FR-17	**Demo mailbox.** Test every email feature without an account	P0	Turning on Demo mailbox lets search, read, draft and send work on sample data; nothing leaves the device	`DemoMailServiceTest`; device
FR-18	**OAuth Gmail connection.** Connect, check and disconnect a Gmail account via OAuth	P0	No password is ever entered in the app; disconnect revokes access	Device
FR-19	**Task management.** Pause, resume, run now, edit and delete tasks	P1	A paused task does not run; "Run now" runs immediately	`TaskExecutionPolicyTest`; device
FR-20	**Readable run results.** Describe task outcomes in plain language with a next step	P1	A failed run explains the cause and what to do; raw text sits behind "Technical details"	`ExecutionLogFormatterTest`
FR-21	**Browser tools for people.** Address bar with search, find in page, reader view, desktop site, text size, bookmarks, history, share	P1	Each control works in the in-app browser	`BrowserUrlTest`; device
FR-22	**AI on web pages.** Summarize, extract key facts, explain or ask about the open page	P1	Tapping Summarize returns to chat with a summary of that page	Device
FR-23	**Theme.** Choose System, Light or Dark	P1	The choice applies immediately and persists	Device
FR-24	**Onboarding and shortcuts.** First-run walkthrough, launcher shortcuts, share text to Aurviz	P2	Onboarding appears once; long-pressing the icon lists shortcuts; sharing text opens a summary	Device
FR-25	**Settings and permissions.** Manage key, default recipient, spoken replies, Gmail and system permissions	P1	All settings persist across restarts	Device

4. Non-functional requirements

ID	Category	Requirement	Measure and result
NFR-01	Performance	Simple commands respond within about 3 s, excluding external service time	On-device overhead about 0.1 s median; end-to-end median 1.6 s over 30 samples (Performance)
NFR-02	Security	Protect keys, tokens and accounts	OAuth only (no passwords); refresh tokens AES-256-GCM encrypted on the server; session token and API key in Keystore-backed encrypted storage; excluded from backup; confirmation before send; no secrets in git
NFR-03	Reliability	Failures never look like successes; state survives failures	Failed runs are recorded as failed; transient errors retry with exponential backoff (up to 3); ambiguous email delivery is stored as UNKNOWN and never auto-resent
NFR-04	Accuracy	Choose the correct tool and clarify ambiguity	Draft-versus-send rule; asks for a missing recipient address; never invents email content
NFR-05	Maintainability	Modular, easy to extend	Layered packages; new tool needs a class, a declaration and one registration line (Architecture)
NFR-06	Scalability	Support more tools and users without redesign	`AgentTool` and `MailService` abstractions; backend is stateless with a PostgreSQL job queue
NFR-07	Fault tolerance	Handle outages gracefully	Offline check before background runs; retry actions; model fallback chain; backend cold-start-safe timeouts and warm-up
NFR-08	Usability	Intuitive interface	One-tap mic, live "what the agent is doing" status, tool timeline, approval dialogs, onboarding, dark mode
NFR-09	Resource efficiency	Low battery and memory impact	WorkManager only (no persistent service), network constraints, bounded tables and context
NFR-10	Compatibility	Wide device support	minSdk 26, adaptive and themed icon, runtime permissions, tested on Android 16
NFR-11	Privacy	Minimise data exposure	Page and text attachments are sent only when the user triggers an action; demo mode never contacts Gmail

5. External interfaces

Interface	Direction	Purpose
Gemini REST `generateContent`	App to Google	Reasoning and function calling, with model fallback
Aurviz backend REST (HTTPS)	App to server	OAuth session, Gmail search/read/draft/send, scheduled email
Gmail API	Server to Google	Mailbox operations with the user's consent
Android SpeechRecognizer / TextToSpeech	App to OS	Voice in and out
Android WorkManager / Notifications	App to OS	Background execution and result alerts

Backend endpoints are listed in API.

6. Data requirements

On device (Room): `memories`, `scheduled_tasks`, `task_executions`, `performance_samples`. Chat history is a bounded JSON file (100 messages). Bookmarks and history for the browser are stored locally. **On the server (PostgreSQL):** `GoogleAccount` (encrypted refresh token), `AppAuthCode`, `DeviceSession` (hashed tokens), `ScheduledEmail`, `EmailExecution`, plus the pg-boss job tables.

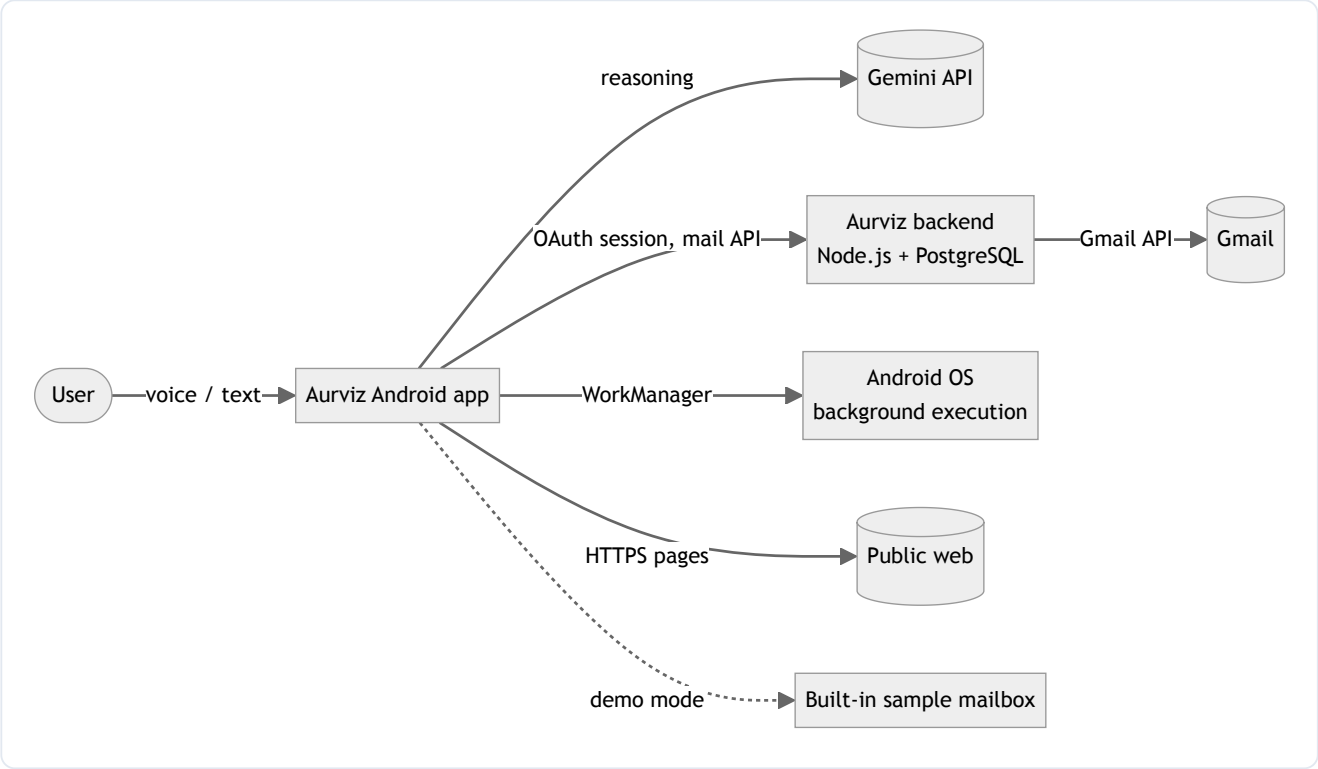
7. Traceability

See the matrix in Test Report.

Architecture

Aurviz is an Android app (Kotlin, Jetpack Compose) plus a small Node.js backend. This document explains how the pieces fit together, why they are separated the way they are, and how to extend the system.

1. System context



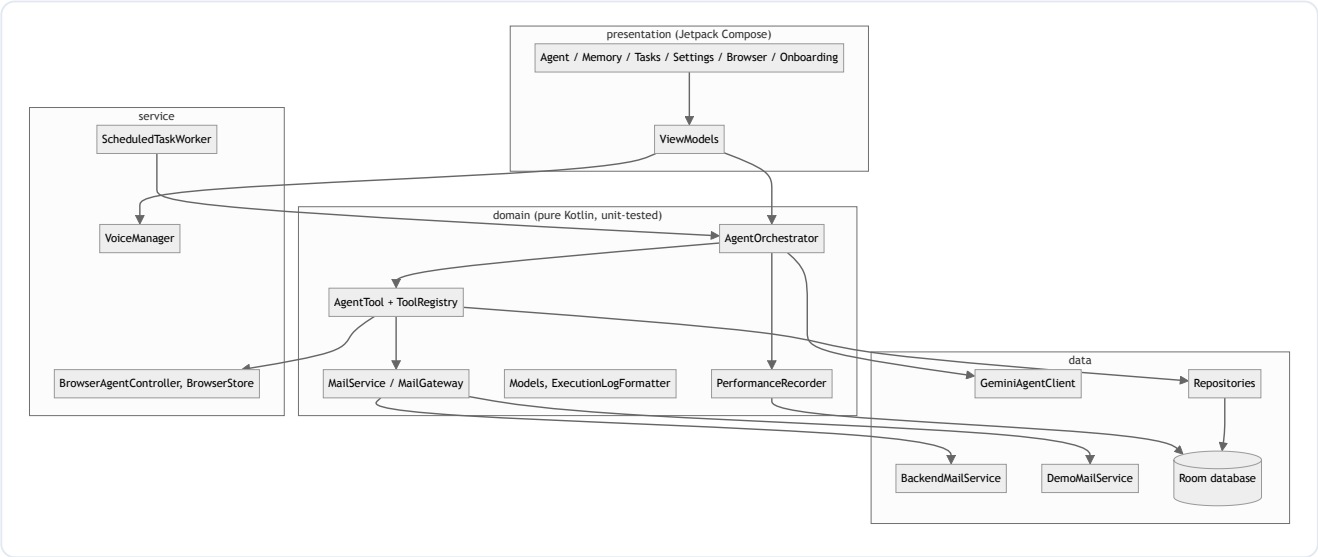
2. Three separate concerns

The most important design decision: **thinking, doing and timing are three different things owned by three different components.**

Concern	Owner	Where in code
Reasoning: decide the next step	Gemini via function calling	`GeminiAgentClient`, prompt in `AgentOrchestrator`
Execution: perform the operation	Kotlin `AgentTool`s and the backend	`data/tools/*`, `backend/src/routes/*`
Scheduling: run later	Android WorkManager (persists its own state)	`TaskSchedulerRepositoryImpl`, `ScheduledTaskWorker`

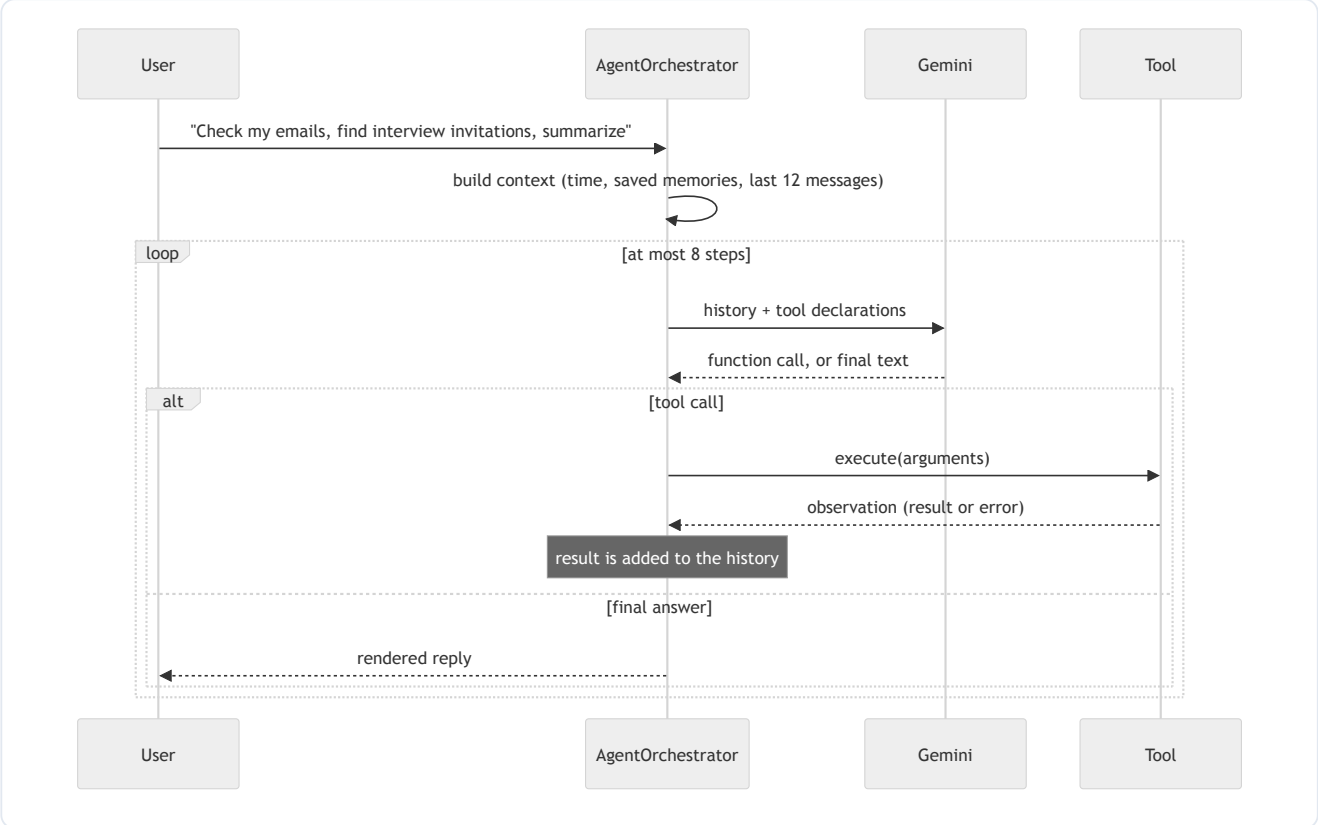
Gemini cannot wake an app. A scheduled task is stored in Room and registered with WorkManager. When WorkManager runs the worker, the worker calls the same agent loop in **background mode**, which has stricter safety rules (see section 6).

3. Layers of the Android app



Rules that keep it maintainable: - `domain` never imports Android UI classes, so the orchestrator is covered by plain JVM tests with fake clients. - Tools depend on interfaces (`MailService`, repositories), not on concrete networking classes. - The UI observes `StateFlow`s and holds no business logic.

4. The agent loop



Behaviours worth knowing: - **Planning rules** are in the system instruction: one tool at a time, search then read then summarize, draft versus send, ask when the recipient is ambiguous, never invent mail content. - **Memory** (lexical match plus the most recent entries) is injected on every turn, so a saved preference like "formal emails" affects later drafts. - **Step limit:** a request cannot loop forever. Reaching the limit shows an explicit message with a retry action. - **Empty answers:** if the model returns no real text (for example a lone dash) the orchestrator asks once more, then

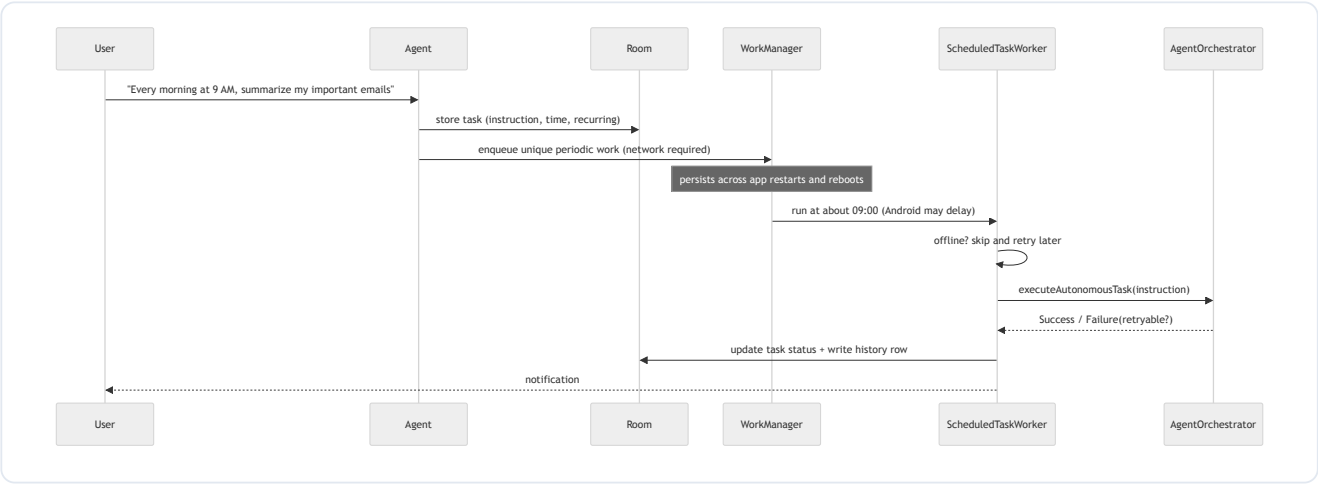
falls back to a retryable error. - **Attachments:** the browser and share-intent can pass page or text content to the model. It is sent only for the action the user triggered and is marked as untrusted data, which reduces prompt-injection risk.

5. Tools and the approval gate

Tool	Side effect	Needs approval
`search_emails`, `read_email`	none	no
`create_email_draft`	saves a draft, sends nothing	no
`send_email`	sends mail	**yes**
`manage_memory` (save, recall, update, delete)	local data	no
`schedule_task`	registers a background job	no
`browse_web`, `open_url`, `read_page`, `scroll_page`, `extract_links`, `navigate_*`	none	no
`click_element`, `enter_text`	can change a web page	**yes**

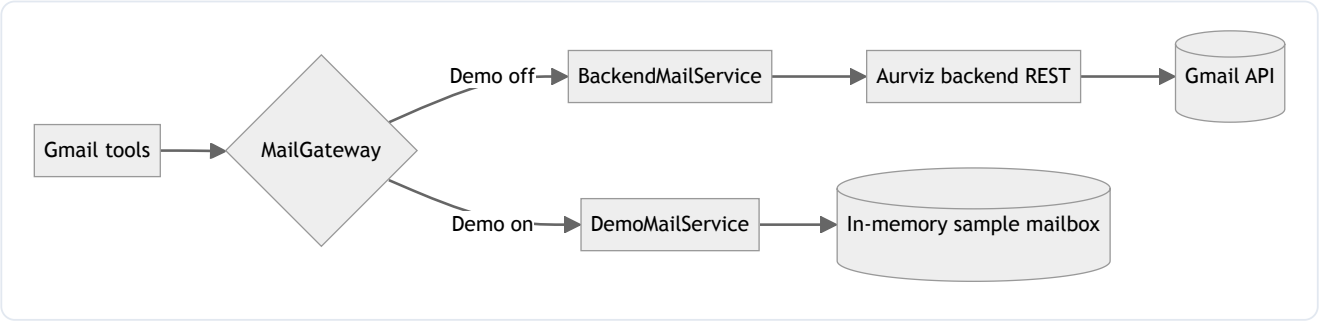
A tool marked `isSensitive` pauses the loop and shows an editable confirmation dialog. Only approval calls `executeConfirmed`. In background runs a sensitive tool is never executed at all; the model is told it was blocked and returns a draft or summary instead.

6. Scheduled tasks



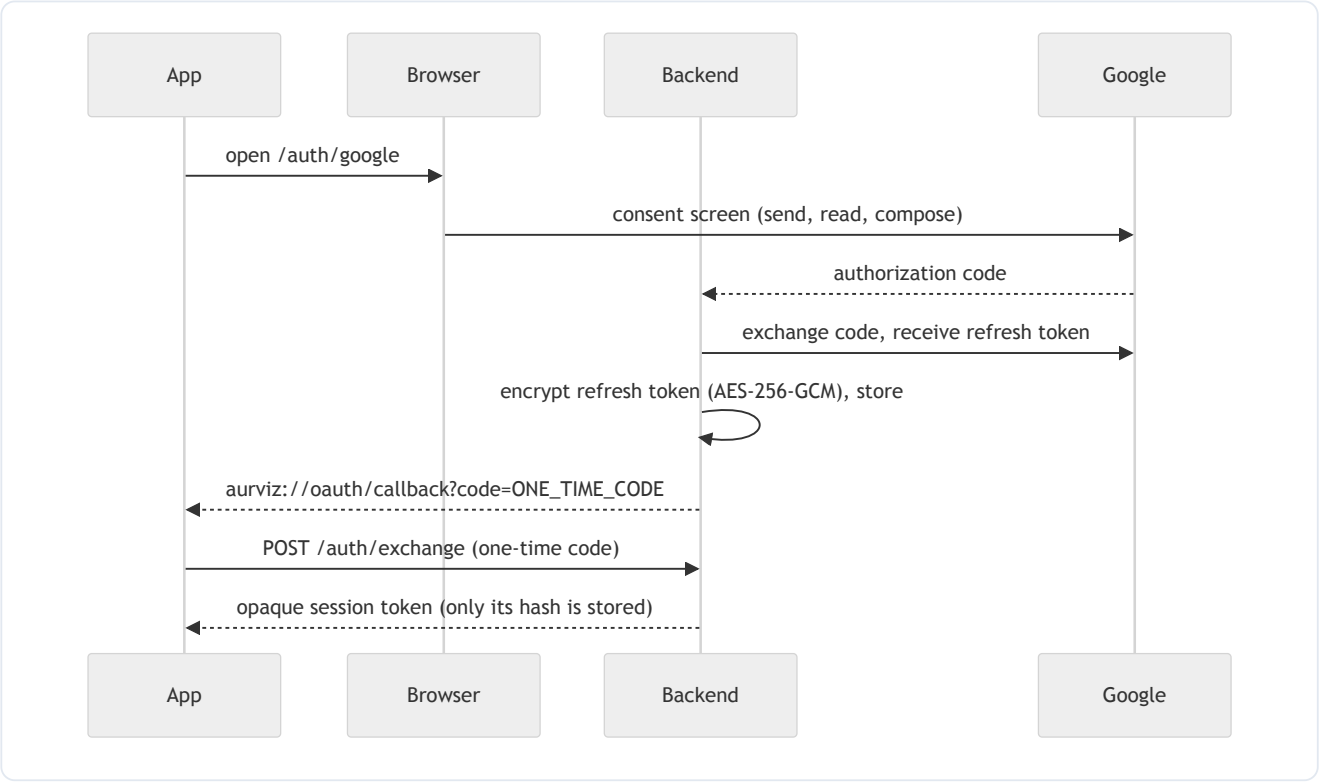
Reliability rules: - Unique work names (`task_<id>`, `task_<id>_manual`) stop duplicate enqueues; an in-process guard drops a concurrent second run. - A run is **never reported as success** if a required tool failed. Transient failures (offline, rate limit, tool outage) are retried with exponential backoff up to 3 attempts and shown as "Retrying". - Pause cancels the WorkManager job but keeps the task; resume re-registers it. A run that was already in flight never silently resumes a paused task. - Every attempt is stored in `task_executions` and shown in Tasks, History. Raw errors are translated by `ExecutionLogFormatter` into a plain explanation with a next step.

7. Mail: real account or demo



`MailGateway` picks the implementation on every call, so the Demo switch takes effect instantly. The demo service understands the same Gmail-style operators (`subject:` , `from:` , `is:unread` , `newer_than:3d` , quoted phrases), keeps drafts and sent mail in their own folders, and makes `send` idempotent like the real backend. Nothing in demo mode contacts Gmail.

Real accounts use OAuth through the backend:



The app never sees a Google password or refresh token. Because the refresh token lives on the server, scheduled email summaries work while the phone is idle.

8. Web browser

`BrowserAgentController` is a single shared session between the agent's browser tools and the visible WebView. People get an address bar that searches when you type words, find in page, reader view, desktop site, text size, bookmarks, history and share. Four AI actions (Summarize, Key facts, Explain simply, Ask) read the page text and hand it to the agent.

Safety: HTTPS only; private, local and link-local addresses are rejected (DNS is resolved for agent opens); no JavaScript bridge to the app; file and content access disabled; Safe Browsing on; clicks and typing by the agent need approval and never submit forms. This is a controlled WebView, not a hardened sandbox.

9. Data

Store	Technology	Content
`memories`	Room	key, content, category, timestamp
`scheduled_tasks`	Room	instruction, schedule, enabled, last status
`task_executions`	Room	per-run status, message, duration, attempt (500 rows kept)
`performance_samples`	Room	latency samples (1000 rows kept)
Chat	JSON file	last 100 messages
Secrets	EncryptedSharedPreferences	Gemini key, session token
Preferences	SharedPreferences	theme, demo mode, onboarding
Server	PostgreSQL + Prisma, pg-boss	accounts (encrypted token), sessions (hashed), scheduled emails, history, job queue

Database upgrades use explicit migrations (`MIGRATION_1_2`), so memories and tasks are never wiped on update.

10. Security

Threat	Mitigation
Stolen credentials	No passwords; OAuth with server-held refresh token (AES-256-GCM)
Token on device	Keystore-backed encrypted preferences, excluded from backup
Unwanted sending	Approval dialog for every send; background runs cannot send
Header injection in mail	Recipient and subject reject line breaks (app and server)
Replay or double send	Idempotency key per request; unique database constraint
Abuse of the API	Rate limiting, Helmet headers, 32 KB body limit, schema validation
Malicious web content	HTTPS only, private-address blocking, no JS bridge, content treated as untrusted data
Secrets in source	None committed; build-time Gemini key is read from the git-ignored `local.properties`

Known limit: a Gemini key bundled into an APK can be extracted from the file. For production, route Gemini through the backend with quotas.

11. Extending Aurviz

Adding a capability, for example Google Calendar, needs **no change to the orchestrator**: 1. Implement `AgentTool` (and a backend route if it needs Google credentials). 2. Add its function declaration in `GeminiAgentClient` . 3. Register it in `AurvizApp` . 4. Set `isSensitive = true` if it has external side effects: it then automatically gets the approval gate and is blocked in background runs.

Swapping the mail provider means implementing `MailService` once; the tools and orchestrator are untouched, which is exactly how the demo mailbox was added.

12. Decisions and trade-offs

Decision	Reason	Trade-off
Backend holds Google tokens	Scheduled mail works while the app is closed; no tokens on the phone	Needs a hosted service; free tier can sleep (the app warms it up and allows a long first request)
Gemini function calling	Reliable structured tool use	Depends on the model API; fallback chain mitigates model retirement
WorkManager for scheduling	Persistent, battery-aware	Not exact to the minute
Lexical memory search	Simple, fast, private	Not semantic; a vector index is a possible upgrade
Demo mailbox	Lets anyone evaluate without credentials	Sample data only

Test Report

Build under test: Aurviz 1.1 debug build, commit history on `main` **Date:** 13 November 2026 **Device:** realme RMX5116, Android 16 (API 36), Wi-Fi **Backend:** deployed instance (`/health` OK), Gmail test account in OAuth testing mode

1. Summary

Area	Result
Android unit tests	**83 of 83 pass**
Backend tests	**14 of 14 pass** (Node 22)
Device checks	**All listed in section 3 pass**
Build	`assembleDebug` succeeds; CI runs tests and the build on every push
Dependency audit	`npm audit` reports no known vulnerabilities

Not verified on the device (stated openly): speech input in the latest build, offline Retry behaviour (unit-tested only, because switching Wi-Fi off would disconnect the wireless debugging link), and the real 09:00 trigger firing unattended overnight.

2. Automated tests

2.1 Android (JVM, `./gradlew app:testDebugUnitTest`)

Suite	Tests	What it proves
`AgentOrchestratorTest`	8	Direct answers, conversation persistence, function-call id round-trip, multi-step background runs, sensitive tools never run in the background, approval flow with edited fields
`AgentReliabilityTest`	9	Real errors with retry (no generic greeting), step limit never silent, multi-step email workflow, preference injected into the prompt, transient tool failure is retryable, latency recorded, empty model answer retried then reported
`GeminiAgentClientTest`	11	Request building, model fallback, error classification, response parsing, thought-signature preservation
`TaskExecutionPolicyTest`	11	Success, skip, retry and failure decisions; recurring tasks stay registered; backoff limits
`TimeUtilsTest`	6	Natural-language time parsing and next-run delay
`DemoMailServiceTest`	6	Gmail-style operators, ordering and caps, read and 404, drafts and sent folders, idempotent send
`ExecutionLogFormatterTest`	6	Raw errors become plain language with a next step
`MemoryToolTest`	4	Upsert by key, update, delete, ambiguous delete deletes nothing, recall
`EmailTaskContextTest`	4	Email-task detection and safe instructions
`BrowserUrlTest`	5	Domain, http upgrade, search fallback, blank input, host display
`GmailInboxToolsTest`	3	Result formatting, no invented data, transient versus permanent error classification
`ControlledWebScraperTest`	3	Public-page extraction and private-address blocking
`MarkdownParserTest`	3	Bold, italic, code, lists, malformed markers

Suite	Tests	What it proves
`SchedulerToolTest`	2	The model's instruction is stored unchanged; invalid time rejected
`MetricMathTest`	2	Percentiles and summary statistics
Total	**83**	

2.2 Backend (`npm test`)

Suite	Tests	What it proves
`gmail-inbox.test.js`	7	MIME body extraction, HTML-to-text, header lookup, raw draft building and header-injection rejection, Google error mapping, result cap, disconnected-account error
`validation.test.js`	3	Recipient, time zone and offset validation; injection rejected
`token-vault.test.js`	2	Refresh-token encryption and tamper detection
`worker.test.js`	2	Durable queue provisioning
Total	**14**	

2.3 Database migration

The `MIGRATION_1_2` SQL was compared against the schema Room generates for the new tables. They match exactly. On a physical device an upgrade install kept the existing chat history and memories.

3. Device test log

Tested on the physical device against the deployed backend.

#	Requirement	Steps	Expected	Result
D1	FR-18	Settings, Connect Gmail, choose account, grant	Connected as the account	Pass
D2	FR-14	Ask for email with an old session that lacks the inbox scope	Clear "reconnect Gmail" message, no invented data	Pass
D3	FR-03, 04, 12, 15	"Check my emails from the last 7 days and summarize them"	`search_emails` runs; summary of real messages	Pass
D4	FR-03, 16	"Draft an email to my professor asking for an extension. Do not send it."	Asks for the missing address, then saves a formal draft, nothing sent	Pass
D5	FR-08	"Send an email to the test account…"	Approval dialog; after approval the mail is delivered	Pass (found by a follow-up inbox search)
D6	FR-06	Saved preference "formal emails", then drafted	Draft is formal	Pass
D7	FR-05	Save a memory, force-stop, relaunch	Memory still listed	Pass
D8	FR-07	Delete a memory by voice command and from the screen	Removed; screen asks for confirmation	Pass
D9	FR-10, 11, 13	"Every morning at 9 AM…", then Run now	Daily task created; Success row in History; notification	Pass
D10	FR-09	"Open news.ycombinator.com and tell me the top 3 stories"	Browser opens, top stories listed	Pass

#	Requirement	Steps	Expected	Result
D11	FR-17	Demo mailbox on, "find interview invitations and summarize"	Search and read run on sample data; accurate summary	Pass
D12	FR-21	Address bar with a domain; browser controls	Page loads; controls respond	Pass
D13	FR-22	Open a Wikipedia article, tap Summarize	Returns to chat with a structured summary of that page	Pass
D14	FR-23	Theme selector System, Light, Dark	Applies immediately	Pass
D15	FR-24	First launch	Splash animation then onboarding	Pass
D16	FR-25	Change the default recipient email and save	New value shown and kept	Pass
D17	NFR-01	30 simple prompts	Median 1.6 s, on-device overhead about 0.1 s	Pass

Defects found during device testing, and fixed

Defect	Fix
Scheduler rewrote the model's instruction by keyword matching ("summarize" became "draft responses")	Instruction now stored unchanged; regression test added
Background email tasks told the model the wrong account address	Removed the claim; the tools use the connected account
A one-off network error left the Gmail status stuck on screen	Status re-checks on opening Settings; Retry button added
Model sometimes returned a lone "-" as its final answer	Token limit raised; one automatic retry; then a retryable error
Cold-start of the free-tier backend caused "could not reach"	Warm-up request at launch; timeouts allow a 45 s wake-up
Status-bar text invisible on light backgrounds	Icons now follow the active theme

4. Requirement-to-evidence matrix

Requirement	Automated	Device
FR-01 Voice input	n/a (platform API)	Earlier build
FR-02 Text input	`AgentOrchestratorTest`	D3
FR-03 Intent detection	`AgentReliabilityTest`	D3, D4
FR-04 Tool calling	`AgentOrchestratorTest` , `GeminiAgentClientTest`	D3
FR-05 Persistent memory	`MemoryToolTest`	D7
FR-06 Memory retrieval	`AgentReliabilityTest` , `MemoryToolTest`	D6

Requirement	Automated	Device
FR-07 Memory edit/delete	`MemoryToolTest`	D8
FR-08 Send after confirmation	`AgentOrchestratorTest`	D5
FR-09 Web automation	`ControlledWebScraperTest`, `BrowserUrlTest`	D10
FR-10 Scheduling	`SchedulerToolTest`, `TimeUtilsTest`	D9
FR-11 Background execution	`TaskExecutionPolicyTest`	D9 (Run now through WorkManager)
FR-12 Multi-step	`AgentReliabilityTest`	D3, D11
FR-13 Task history	n/a	D9
FR-14 Error reporting	`AgentReliabilityTest`, `ExecutionLogFormatterTest`	D2
FR-15 Search and read	`DemoMailServiceTest`, `gmail-inbox.test.js`	D3, D11
FR-16 Drafts	`GmailInboxToolsTest`	D4
FR-17 Demo mailbox	`DemoMailServiceTest`	D11
FR-18 OAuth	`token-vault.test.js`	D1
FR-19 Task management	`TaskExecutionPolicyTest`	D9
FR-20 Readable results	`ExecutionLogFormatterTest`	Screen check
FR-21 Browser tools	`BrowserUrlTest`	D12
FR-22 AI on pages	n/a	D13
FR-23 Theme	n/a	D14
FR-24 Onboarding, shortcuts	n/a	D15
FR-25 Settings	n/a	D16
NFR-01 Performance	`MetricMathTest`	D17
NFR-02 Security	`token-vault.test.js`, `validation.test.js`	D1, D5
NFR-03 Reliability	`TaskExecutionPolicyTest`, `AgentReliabilityTest`	D9
NFR-07 Fault tolerance	`AgentReliabilityTest`	D2

5. How to reproduce

```
./gradlew app:testDebugUnitTest        # 83 Android tests
./gradlew app:assembleDebug            # build the APK
cd backend && npm ci && npm test       # 14 backend tests
```

Device checks follow the Evaluator Guide.

Performance

Method

Aurviz measures itself. `PerformanceRecorder` writes one row per sample into Room (newest 1000 kept), and **Settings, Performance** shows count, median, 95th percentile and success rate per metric.

Metric	What is timed
Agent turn (end-to-end)	Prompt submitted until the answer or error is shown. Includes Gemini and Gmail network time, and any time spent waiting on an approval dialog
Gemini call	One `generateContent` round trip, including model fallback
Memory retrieval	The Room lookup that builds the memory context
Tool: `<name>`	One tool execution, for example `search_emails`
Scheduled task run	One background WorkManager run, start to persisted result

Percentiles use the nearest-rank method (`MetricMath`, unit-tested).

Results (13 November 2026)

Device: realme RMX5116, Android 16 (API 36), Wi-Fi, debug build, model `gemini-3.5-flash-lite`. **Workload:** 30 simple one-line prompts needing no tools, sent through the UI after resetting the samples.

Metric	n	Median	p95	Success
Agent turn (end-to-end)	30	1.6 s	6.6 s	100%
Gemini call	30	1.5 s	6.4 s	100%
Memory retrieval (Room)	30	54 ms	93 ms	100%

Reading the numbers against the target

The target is under 3 s for simple commands, excluding external service time.

- **On-device overhead is about 0.1 s at the median** (agent turn minus Gemini call). That covers memory retrieval, prompt assembly, tool dispatch, persistence and UI state.
- **End-to-end median is 1.6 s**, below 3 s even including the network round trip.
- **The 6.6 s p95 is almost entirely Gemini's own latency** (its p95 is 6.4 s): external service time, not on-device work.

Other single observations: `search_emails` about 0.9 s for one call, `manage_memory` 43 ms, and a manually triggered background run that searched Gmail and summarized took 9.9 s end-to-end (several model and tool steps, each adding a Gemini round trip).

Limitations of this study

One device, one network, one time of day, simple prompts. Multi-step email workflows take proportionally longer. No battery or long-run memory study was performed. WorkManager timing is not exact, so scheduled-run delay was not benchmarked.

Reproduce

Settings, Performance, Reset samples, send at least 30 prompts, read the table.

Resource-efficiency choices

- No foreground or persistent background service. Deferred work uses WorkManager with a network constraint, so it never wakes the device just to fail offline.
- Bounded context: 12 prior messages, at most 8 memories, email bodies capped at 6,000 characters, search capped at 15 results, at most 8 interactive and 6 background model steps.
- Bounded tables: performance 1000 rows, task history 500 rows, chat 100 messages, browser history 60 pages.
- The hosted backend is warmed at launch to hide cold starts.

Evaluator Guide

This guide lets you test every feature of Aurviz in about ten minutes **without anyone's password or Google account**.

1. Install

1. Install the APK on an Android 8.0+ phone (allow "install unknown apps" for your file manager or browser).
2. Open **Aurviz**. You will see the animated logo, then a short walkthrough.
3. On the last page choose **Start with the demo mailbox**. (You can switch later in Settings, Gmail, Demo mailbox.)
4. Allow the microphone and notification permissions when asked.

The AI key is bundled in the build, so there is nothing to configure. If your build has no bundled key, open Settings, AI model and paste a free Gemini key from https://aistudio.google.com/.

2. What the demo mailbox contains

Ten realistic sample emails: an interview invitation, a campus placement drive, a reply from a professor about a deadline, an application acknowledgement, a GitHub notification, a bank security alert, a friend asking about the weekend, job alerts, a newsletter and an invoice. Nothing you do in demo mode is ever sent to a real address.

3. Ten-minute test script

Type each prompt (or tap the mic and say it). The badge **DEMO** in the header confirms demo mode.

#	Say or type	What you should see	Requirement
1	"Check my emails, find interview invitations and summarize them"	Cards "Search Gmail" then "Read email", then a structured summary with the Monday 10:30 interview	FR-03, 04, 12, 15
2	"Which of my emails are important and unread?"	Placement drive and interview invitation	FR-15
3	"Remember that I prefer formal emails"	Memory saved card; the Memory tab lists it	FR-05
4	"Draft an email to Prof. Rao asking about my grade, but don't send it"	It asks for the address if unknown, then saves a **formal** draft; nothing is sent	FR-06, 16
5	"Find my drafts"	The draft you just saved (`in:drafts`)	FR-16
6	"Send an email to rahul@example.com saying I will be late"	An **approval dialog** with editable To, Subject, Body. Cancel sends nothing; approve records it in the demo outbox	FR-08
7	"Every morning at 9 AM, check my important emails and summarize them"	Task created. Open **Tasks**, tap **Run now**; a notification arrives and **History** shows the run	FR-10, 11, 13
8	"Forget my email tone preference"	Memory deleted	FR-07
9	"Open news.ycombinator.com and tell me the top 3 stories"	The in-app browser opens and the stories are listed	FR-09
10	In the browser open any Wikipedia article, tap **Summarize**	Back in chat with a summary of that page	FR-22

4. Other things to look at

- **Settings, Appearance:** System, Light and Dark themes.
- **Browser menu (three dots):** Find in page, Reader view, Desktop site, text size, Share, Copy link, Open in Chrome. Star button for bookmarks; library button for bookmarks and history.
- **Tasks:** pause and resume a daily task; the History tab explains failures in plain words with a next step.
- **Settings, Performance:** latency measured on your device (median and p95).
- **Long-press the app icon:** shortcuts for Voice command, Summarize inbox and Tasks.
- **Share text** from any app to Aurviz to get a summary.
- **Failure handling:** switch on airplane mode and send a prompt. You get a clear message and a **Try again** button.

5. Testing with a real Gmail account (optional)

Google only allows listed accounts to grant the Gmail read scope while an app is in testing mode. Two ways to run the live path:

1. **Add the evaluator's account as a test user** (project owner, one minute): Google Cloud Console, APIs and Services, OAuth consent screen, Audience, Test users, Add users. The evaluator then taps Settings, Connect Gmail, picks their own account, and approves. They never share a password.
2. **Publish the consent screen "In production"** (unverified). Any Google user can then continue through the "Google hasn't verified this app" warning (Advanced, Go to Aurviz), limited to 100 users until Google verifies the app.

In both cases the evaluator signs in on Google's own page. Aurviz never sees the password.

6. Mapping to common evaluation criteria

Criterion	Where to look
AI reasoning and tool calling	Script 1, 4, 6; the tool cards in chat; Architecture
Modular architecture	Architecture, the `domain`, `data`, `presentation`, `service` packages
Persistent memory	Script 3, 4, 8; restart the app and check the Memory tab
Background scheduling	Script 7; Architecture, scheduled tasks
OAuth and secure email	Section 5 above; Security
Error handling and reliability	Airplane-mode test; Tasks, History; Test Report
Performance measurement	Settings, Performance; Performance
UI and UX	Whole app, dark mode, onboarding, animations

User Guide

Talking to Aurviz

Type in the message box or tap the microphone. While Aurviz works, the header shows what it is doing ("Searching your inbox…"). Every tool it uses appears as a card, so you can see exactly what happened.

Examples: - "Summarize my unread emails from today" - "Draft a reply to the interview invitation, but don't send it" - "Remember that my manager is Alex and her email is alex@company.com" - "Every weekday morning at 8, summarize my inbox" (daily recurrence; weekday-only rules are not supported) - "Open the BBC News website and tell me the top story"

Email

- **Search and read:** Aurviz uses Gmail search syntax internally (`from:` , `subject:` , `is:unread` , `newer_than:3d`).
- **Drafts:** "draft…" or "don't send" always saves a draft.
- **Sending:** Aurviz shows **To, Subject and Body** in an editable dialog. Nothing is sent until you tap **Send email**. Scheduled tasks can never send.
- **Connect your account:** Settings, Gmail, Connect Gmail. Sign-in happens on Google's page.
- **Try without an account:** Settings, Gmail, Demo mailbox.

Memory

Aurviz remembers facts and preferences and uses them in later answers. The **Memory** tab lets you search, add, edit and delete. Deleting asks for confirmation. You can also say "forget my…".

Tasks

The **Tasks** tab lists routines. For each task you can **Run now**, **pause or resume** (daily tasks), **edit** or **delete**. The **History** tab keeps every run with the time taken. Tap a task's "View last result" to read what happened. If something failed you get a plain explanation and what to do next, with technical details one tap away.

Android may start a scheduled task a few minutes late to save battery.

Browser

Tap the globe in the chat header. - Type a website or just words to search. - Menu (three dots): Find in page, Reader view, Desktop site, Larger or smaller text, Share link, Copy link, Open in Chrome. - Bottom bar: back, forward, home, bookmark star, bookmarks and history. - **AI chips:** Summarize, Key facts, Explain simply, Ask about this page. The page content is sent to Aurviz only when you tap one of these.

Settings

- **Appearance:** System, Light or Dark.
- **Gmail:** connect, disconnect or use the demo mailbox.
- **AI model:** your own Gemini key (optional if the build includes one).
- **Preferences:** default recipient and spoken replies.
- **Permissions:** shortcut to Android's permission page.

- **Performance:** measured response times.

Shortcuts

Long-press the Aurviz icon for Voice command, Summarize inbox and Tasks. Share text from another app and choose Aurviz to get a summary.

Troubleshooting

Problem	What to do
"Could not reach Aurviz"	The hosted server may be waking up. Wait a few seconds and tap Retry
"Gmail inbox access was not granted"	Settings, Disconnect, then Connect Gmail again and approve all permissions
"Gemini API key is not configured"	Add a key in Settings, AI model
Task ran late	Normal Android battery behaviour; use Run now for immediate execution

Backend API

Base URL is the deployed HTTPS service. All `/api/v1` calls except the OAuth exchange need `Authorization: Bearer <session token>`. Errors return `{ "error": "message" }`.

Health

Method	Path	Notes
GET	`/health`	`{"status":"ok"}` when the database and job queue are reachable

Authentication

Method	Path	Description
GET	`/auth/google`	Starts Google OAuth (adds CSRF state), redirects to Google
GET	`/auth/google/callback`	Exchanges the code, stores the encrypted refresh token, redirects to `aurviz://oauth/callback?code=<one-time code>`
POST	`/api/v1/auth/exchange`	Body `{ "code": "<one-time code>" }`, returns `{ accessToken, expiresAt }`
GET	`/api/v1/auth/me`	Connected account and expiry
POST	`/api/v1/auth/session/refresh`	Rotates the session token
DELETE	`/api/v1/auth/session`	Revokes this session
DELETE	`/api/v1/auth/google`	Disconnects the account, cancels pending sends, revokes the Google token

Mailbox

Method	Path	Description
GET	`/api/v1/gmail/messages?q=&max=`	Search with Gmail syntax. `max` 1 to 15. Returns id, from, to, subject, date, snippet, unread
GET	`/api/v1/gmail/messages/:id`	One message as plain text (HTML converted, attachments skipped, 6,000 character cap)
POST	`/api/v1/gmail/drafts`	Body `{ recipient, subject, body }`. Saves a draft, sends nothing

Example:

```
GET /api/v1/gmail/messages?q=subject:interview%20newer_than:7d&max=5
Authorization: Bearer <token>
```

```
{ "query": "subject:interview newer_than:7d", "messages": [
  { "id": "19a1d…", "from": "HR <hr@example.com>", "subject": "Interview invitation",
    "date": "Thu, 12 Nov 2026", "snippet": "Please confirm…", "unread": true } ] }
```

Sending and scheduling

Method	Path	Description
POST	`/api/v1/emails/send`	Queue an immediate email. Header `Idempotency-Key` (8 to 120 safe characters) required. Returns 202
POST	`/api/v1/emails/scheduled`	Schedule one email for a future time (at least 30 s ahead)
GET	`/api/v1/emails/scheduled`	List scheduled emails with recent history
GET	`/api/v1/emails/scheduled/:id`	One email with execution history
PATCH	`/api/v1/emails/scheduled/:id`	Update a pending email (stale queued jobs cannot send the old version)
DELETE	`/api/v1/emails/scheduled/:id`	Cancel a pending email

Delivery semantics

Gmail acceptance and the database commit cannot be atomic. Definite 4xx failures become `FAILED`; quota (429) is retried with backoff; network or 5xx outcomes become `UNKNOWN` and are **never** resent automatically. The same idempotency key never produces two messages.

Security

HTTPS, CSRF state on OAuth, one-time codes, hashed session tokens, AES-256-GCM refresh tokens, Helmet, rate limiting (120 requests per minute), 32 KB body limit, schema validation, header-injection rejection, redacted logs.

Build and Deployment

1. Build the Android app

Requirements: Android Studio, JDK 17, Android SDK 34.

```
./gradlew app:testDebugUnitTest      # run the tests
./gradlew app:assembleDebug          # APK at app/build/outputs/apk/debug/app-debug.apk
```

Bundle a Gemini key for evaluators (optional, recommended)

Create `local.properties` (it is git-ignored) and add:

```
sdk.dir=/path/to/Android/sdk
gemini.api.key=YOUR_GEMINI_KEY
```

Build again. The APK then works with no setup. A user's own key entered in Settings always takes priority. **Never commit** `local.properties`. A key inside an APK can be extracted, so use a free-tier key you can rotate, and restrict it in Google AI Studio.

The API base URL is `API_BASE_URL` in `app/build.gradle.kts`.

2. Backend (Node.js 22.12+, PostgreSQL)

```
cd backend
cp .env.example .env        # fill in the values below
npm ci
npx prisma migrate deploy
npm start
```

Variable	Meaning
`DATABASE_URL`	PostgreSQL connection string
`SESSION_SECRET`	32+ random bytes for the OAuth session cookie
`TOKEN_ENCRYPTION_KEY_B64`	base64 of 32 random bytes, encrypts refresh tokens
`GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`	Web OAuth client
`GOOGLE_REDIRECT_URI`	`https://<host>/auth/google/callback`
`ANDROID_REDIRECT_URI`	`aurviz://oauth/callback`
`PUBLIC_BASE_URL`	Public HTTPS URL of the service
`ROLE`	`all`, `api` or `worker`

Generate secrets with `openssl rand -base64 32`.

3. Google Cloud setup (for real Gmail)

1. Create a project and enable the **Gmail API**.
2. OAuth consent screen: add the scopes `gmail.send`, `gmail.readonly`, `gmail.compose` (plus `openid`, `email`).

3. Add test users (or publish unverified, see the Evaluator Guide).
4. Create an OAuth client of type **Web application** and add the exact HTTPS callback URL.
5. Put the client id and secret in the backend environment (never in the app or the repository).

`gmail.readonly` is a restricted scope: it works for listed test users, and a public release needs Google verification. Accounts connected before the read scopes were added must disconnect and reconnect.

4. Hosting notes

- Any host with PostgreSQL works. The reference deployment is a free web service plus a free database, which sleeps when idle. The app wakes it at launch and allows a 45 s first request.
- Run at least one worker (`ROLE=all` does both API and worker).
- Enable database TLS and backups for production.

5. Continuous integration

`.github/workflows/build.yml` runs the Android unit tests and builds the APK, and runs the backend tests and `npm audit` on every push.